

Introduction à la Data Science et manipulation de données

Python, Pandas, Numpy

-  Comprendre la place de la data science dans le Big Data
-  Revoir les bases de Python appliqué à la donnée
-  Savoir importer, nettoyer et transformer un jeu de données réel

La Data Science dans l'écosystème Big Data



Big Data



Data Science



Value

Les "3V" du Big Data

VOLUME

Quantités massives de données

VELOCITY

Génération et traitement rapides

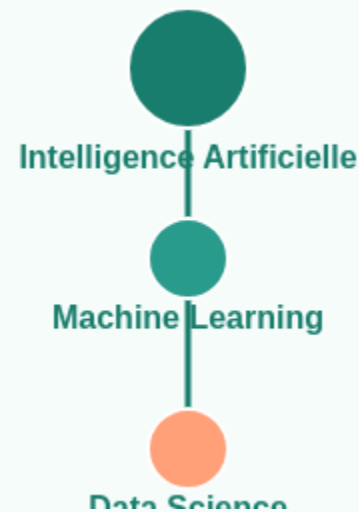
VARIETY

Diversité des formats et sources

Rôle de la Data Science

- ✓ Transforme le Big Data en ressource stratégique
- ✓ Fournit les techniques pour analyser des ensembles de données complexes
- ✓ Permet de passer de la collecte à la compréhension des phénomènes sous-jacents

Définitions et distinctions : Data Science vs Machine Learning vs IA



Intelligence Artificielle

Domaine vaste de l'informatique visant à créer des machines capables de simuler l'intelligence humaine, incluant des capacités comme l'apprentissage, la résolution de problèmes, la perception et la compréhension du langage.



Machine Learning

Sous-ensemble de l'IA qui se concentre sur le développement d'algorithmes permettant aux systèmes d'apprendre à partir de données sans être explicitement programmés.



Data Science

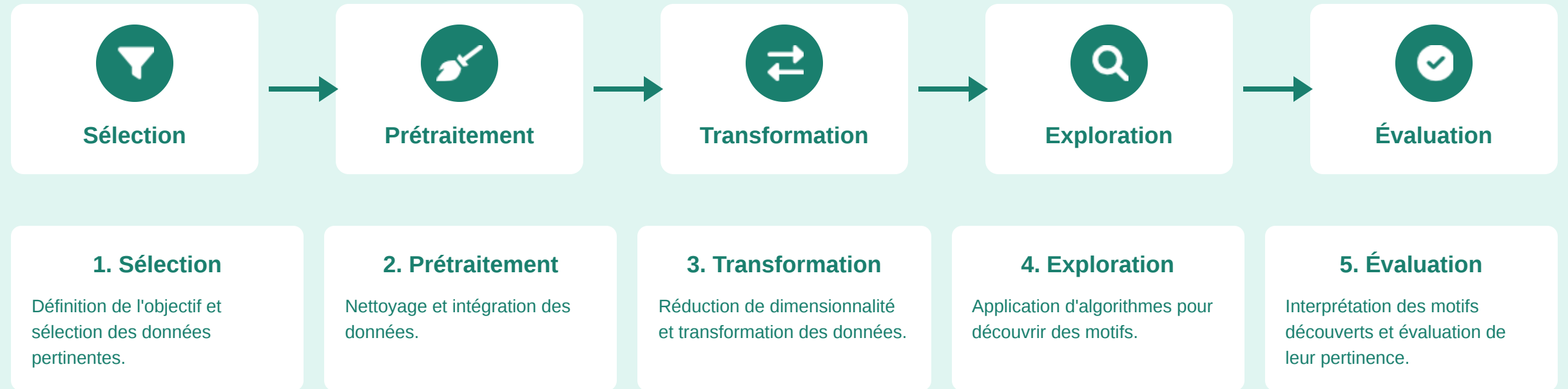
Domaine interdisciplinaire qui combine des méthodes scientifiques, des processus, des algorithmes et des systèmes pour extraire des connaissances de données structurées et non structurées.

Différences fondamentales

- ✓ L'IA vise à simuler l'intelligence humaine, tandis que le ML se concentre sur l'apprentissage à partir de données.
- ✓ La Data Science englobe le ML mais va plus loin par sa focus sur l'ensemble du processus d'analyse des données.

Cycle de vie d'un projet data : KDD

Knowledge Discovery in Databases



i Le processus KDD permet d'extraire des connaissances utiles à partir de données brutes et complexes.

Cycle de vie d'un projet data : CRISP-DM



1. Compréhension métier

Définition des objectifs du projet et des exigences du client.



2. Compréhension des données

Collecte initiale des données et exploration pour identifier les problèmes de qualité.



3. Préparation des données

Nettoyage, transformation et sélection des données.



4. Modélisation

Sélection et application de techniques de modélisation.



5. Évaluation

Évaluation des modèles par rapport aux objectifs métier.



6. Déploiement

Mise en œuvre du modèle et suivi de ses performances.

Cycle de vie d'un projet data : SEMMA

SEMMA est une méthodologie de Data Science qui guide les équipes à travers cinq étapes structurées pour mener à bien un projet data.



Applications et Spécificités

- Surtout utilisée dans le secteur financier et de la santé pour la construction de modèles prédictifs
- Met l'accent sur la qualité de l'échantillon initial et l'exploration approfondie des données
- Permet une validation rigoureuse des modèles à travers l'évaluation

Enjeux métiers et cas d'usage réels



Finance

Enjeu métier:

Minimiser les pertes financières dues aux transactions frauduleuses

Valeur ajoutée:

- Identification rapide des comportements suspects
- Réduction des faux positifs
- Amélioration de la sécurité des transactions



E-commerce

Enjeu métier:

Augmenter les ventes et l'engagement client

Valeur ajoutée:

- Personnalisation de l'expérience d'achat
- Suggestion de produits pertinents
- Augmentation du panier moyen et fidélisation



Santé

Enjeu métier:

Améliorer la précision des diagnostics et optimiser les traitements

Valeur ajoutée:

- Analyse de données médicales complexes
- Identification de patterns et prédiction de maladies
- Médecine personnalisée adaptée aux profils individuels



Marketing

Enjeu métier:

Optimiser les campagnes marketing et maximiser le ROI

Valeur ajoutée:

- Identification de segments de clientèle spécifiques
- Personnalisation des messages publicitaires
- Amélioration de l'efficacité des stratégies de communication



Industrie

Enjeu métier:

Réduire les temps d'arrêt imprévus et les coûts de maintenance

Valeur ajoutée:

- Prédiction des pannes d'équipement avant qu'elles ne surviennent
- Maintenance proactive
- Optimisation de la durée de vie des machines

L'écosystème Python pour la Data Science



Numpy

Numerical Python

- ✓ Fondamentale pour le calcul numérique
- ✓ Objets tableaux multidimensionnels (ndarrays)
- ✓ Fonctions mathématiques rapides sur ces tableaux



Pandas

Manipulation et analyse de données

- ✓ Construite sur Numpy
- ✓ Structures de données Series et DataFrames
- ✓ Facilite l'importation, le nettoyage et la transformation



Matplotlib

Création de visualisations statiques

- ✓ Librairie de base pour les visualisations 2D
- ✓ Génération de graphiques variés (linéaires, nuages de points, histogrammes)
- ✓ Grande flexibilité pour la personnalisation



Seaborn

Visualisations statistiques esthétiques

- ✓ Basée sur Matplotlib
- ✓ Interface de haut niveau pour des visualisations informatives
- ✓ Idéale pour explorer les relations entre variables et visualiser des distributions

Bibliothèques de base

Numpy

Pandas

Visualisation

Matplotlib

Seaborn

Numpy : fondamentaux et applications

Introduction à NumPy

- ✓ Bibliothèque fondamentale pour le calcul numérique en Python
- ✓ Fournit des objets tableaux multidimensionnels (ndarrays)
- ✓ Base de nombreuses autres librairies scientifiques

L'objet ndarray

- ✓ Tableau homogène (tous les éléments sont du même type)
- ✓ Permet des opérations vectorisées très rapides
- ✓ Peut avoir n'importe quel nombre de dimensions

Opérations vectorisées

- ✓ Applications efficaces de fonctions sur des tableaux entiers
- ✓ Élimine les boucles for traditionnelles (plus rapides)
- ✓ Permet des opérations mathématiques vectorielles et matricielles

Exemples de code

```
# Création et opérations avec des tableaux
import numpy as np
# Création d'un tableau 1D
a = np.array([1, 2, 3, 4, 5])
# Création d'un tableau 2D
b = np.array([[1, 2, 3], [4, 5, 6]])
# Opérations vectorisées
c = a * 2 # [2, 4, 6, 8, 10]
d = a + c # [3, 6, 9, 12, 15]
```

 **Avantage principal de NumPy :** performance et simplicité d'utilisation pour les calculs numériques sur des grandes échelles de données.

Pandas : structures de données fondamentales

Series

Structure de données unidimensionnelle (1D) étiquetée.

Caractéristiques:

- Contient des données de types homogènes
- Chaque élément est associé à un index
- Peut être vu comme une colonne dans une base de données
- Facile à manipuler avec des opérations vectorisées

Exemple:

 [25, 30, 35]
 Âges des individus

 *Idéal pour stocker et manipuler une seule colonne de données*

DataFrame

Structure de données bidimensionnelle (2D) mutable et potentiellement hétérogène.

Caractéristiques:

- Collection de Series partageant le même index
- Chaque colonne peut contenir des données de types différents
- Idéal pour représenter des jeux de données tabulaires
- Fonctionnalités puissantes pour l'analyse et la manipulation

Exemple:

Nom	Âge	Ville
Jean	25	Paris
Marie	30	London

 *Parfait pour l'analyse de données structurées provenant de fichiers CSV, Excel ou bases de données*

Types de données en Python pour la Data Science

Types natifs de Python

int

Nombres entiers

```
x = 10
y = -5
```

float

Nombres à virgule

```
x = 3.14
y = -0.5
```

A str

Chaînes de caractères

```
s1 = "bonjour"
s2 = 'Python'
```

bool

Valeurs booléennes

```
is_true = True
is_false = False
```

list

Collections modifiables

```
lst = [1, 2, 3]
lst.append(4)
```

tuple

Collections immuables

```
tpl = (1, 2, 3)
# tpl[0] = 4 → Erreur!
```

dict

Paires clé-valeur

```
dct = {'a': 1, 'b': 2}
print(dct['a'])
```



Structures spécifiques à la Data Science



ndarray (Numpy)

- Tableau multidimensionnel homogène
- Optimisé pour les opérations mathématiques
- Base pour d'autres librairies scientifiques

```
import numpy as np
arr = np.array([1, 2, 3, 4])
arr * 2 # [2, 4, 6, 8]
```



Series (Pandas)

- Tableau unidimensionnel étiqueté
- Peut contenir n'importe quel type de données
- Chaque élément a un index associé

```
import pandas as pd
ser = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
ser['a'] # 1
```



DataFrame (Pandas)

- Tableau de données bidimensionnel
- Colonnes peuvent contenir des types différents
- Ideal pour représenter des jeux de données tabulaires

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df.head()
```

Importation de données avec Pandas



Importation depuis CSV

```
import pandas as pd

# Importer un fichier CSV
df_csv =
pd.read_csv('chemin/vers/mon_fichier.csv')
df_csv.head()
```

Paramètres importants :

- sep : Délimiteur de colonnes (par défaut : ',')
- header : Index de la ligne d'en-tête (par défaut : 0)
- index_col : Index des colonnes (par défaut : None)
- skiprows : Nombre de lignes à ignorer au début



Importation depuis Excel

```
import pandas as pd

# Importer un fichier Excel
df_excel =
pd.read_excel('chemin/vers/mon_fichier.xlsx',
sheet_name='Feuille1')
df_excel.head()
```

Paramètres importants :

- sheet_name : Nom ou index de la feuille à lire
- header : Index de la ligne d'en-tête (par défaut : 0)
- usecols : Colonnes à lire (par index ou nom)
- skiprows : Nombre de lignes à ignorer au début



Conseils :

Toujours vérifier les types de données après l'importation. Utilisez `df.dtypes` pour inspecter les types et `df.info()` pour un aperçu général du DataFrame.

Nettoyage des données : valeurs manquantes et doublons

Valeurs manquantes

 **df.isnull()** : Identifie les valeurs manquantes

 **df.isnull().sum()** : Compte par colonne

 **df.fillna(valeur)** : Remplace les valeurs

 **df.dropna()** : Supprime les lignes/colonnes

```
# Exemple de gestion des valeurs  
manquantes  
if 'Age' in df.columns:  
    df['Age'] =  
df['Age'].fillna(df['Age'].mean())
```

Correction des types

 **df.dtypes** : Affiche les types de données

 **df['colonne'].astype(type)** : Conversion

```
# Exemple de correction des types  
if 'ID' in df.columns:  
    df['ID'] = df['ID'].astype(int)
```

 *Assurez-vous des types avant les opérations*

Doublons

 **df.duplicated()** : Retourne les doublons

 **df.drop_duplicates()** : Supprime les doublons

```
# Exemple de gestion des doublons  
print(f"Avant: {len(df)} lignes")  
df.drop_duplicates(inplace=True)  
print(f"Après: {len(df)} lignes")
```

 **Conseil** : Le nettoyage représente environ 80% du temps d'un projet de Data Science.

Transformation des données

Techniques pour sélectionner, filtrer, créer et transformer des données avec Pandas

Sélection et filtrage

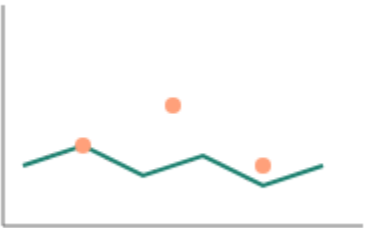
➤ **Sélection de colonnes** : `df[ 'colonne' ]`

Visualisation avec Matplotlib et Seaborn

La visualisation de données est un outil essentiel pour l'exploration et la communication des résultats d'analyse. Elle permet de transformer des informations complexes en représentations visuelles faciles à comprendre.

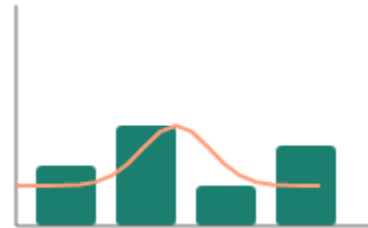
Matplotlib

- ✓ Bibliothèque de base pour la création de visualisations statiques en 2D
- ✓ Permet de générer une grande variété de graphiques
- ✓ Interface de bas niveau pour un contrôle précis



Seaborn

- ✓ Basé sur Matplotlib, offre une interface de haut niveau
- ✓ Crée des visualisations statistiques plus esthétiques et informatives
- ✓ Particulièrement utile pour explorer les relations entre variables



Meilleures pratiques

- 💡 Choisissez le type de graphique adapté à vos données
- 💡 Prénez des visualisations simples et claires
- 💡 Utilisez des couleurs harmonieuses et accessibles
- 💡 Appelez les conclusions directement à partir des visualisations

Conclusion et perspectives

Synthèse des concepts clés



Data Science et Big Data

Positionnement de la Data Science dans l'écosystème du Big Data (3V), transformation des données en ressource stratégique



Écosystème Python

Librairies fondamentales : Numpy, Pandas, Matplotlib, Seaborn pour la manipulation et la visualisation de données



Méthodologies

Approches structurées pour les projets data (KDD, CRISP-DM, SEMMA)



Préparation des données

Importation, nettoyage et transformation des données pour l'analyse

Vers des applications avancées



Visualisation avancée

Techniques de visualisation pour une meilleure compréhension des données



Machine Learning

Applications pour l'extraction de connaissances et la prédiction



Applications métier

Cas d'usage dans la finance, la santé, le marketing et l'industrie

Merci pour votre attention !